

Sistema web de gestión y control de citas para una barbería

Diseño, desarrollo, implementación y evaluación del prototipo

Alumno

Fecha .. de septiembre de 2026

Propósito del documento

Este informe explica cómo funciona el sistema de citas de la barbería y qué resultados obtuvo en las pruebas. Retoma los requisitos de la primera entrega para describir el diseño y la implementación de esta segunda etapa. También sirve como base para presentar y demostrar el proyecto.

Resultado principal

El sistema permite reservar citas, administrar horarios y consultar el historial de cambios. En la prueba local aprobaron los 26 casos ejecutados. El registro muestra 27 resultados porque también cuenta la aprobación del grupo de integración. No hubo fallos.

Índice e introducción

Contenido del informe	Página
Portada y propósito	1
Índice e introducción	2
Diseño de la arquitectura	3
Diseño de procesos y reglas	4
Diseño de la base de datos	5
Diseño de la interfaz	6
Desarrollo e implementación	7
Pruebas y resultados	8
Evaluación de calidad	9
Conclusiones y referencias	10

Introducción

Para dar una cita, la barbería necesita saber quién puede atender y cuánto tiempo requiere el servicio. Si las reservas quedan repartidas entre mensajes y notas, un mismo horario puede asignarse dos veces. También es fácil perder de vista un cambio. La primera entrega describió este problema, las personas que usarían el sistema y sus requisitos.

En esta etapa, la propuesta tomó forma en un sitio web. El cliente puede consultar los servicios, elegir un horario y reservar sin abrir una cuenta. El administrador sí debe iniciar sesión. Desde su panel revisa la agenda, modifica los catálogos y atiende cancelaciones, cambios de horario o ausencias.

La prioridad fue comprobar que las reglas funcionaran al guardar una cita. Por ejemplo, si dos personas eligen el mismo horario, el sistema debe aceptar solo una reserva. Las pruebas de esta entrega revisan ese caso y otras operaciones de la agenda.

Alcance y condiciones

El sistema incluye servicios, barberos, clientes y citas, además de jornadas, descansos, bloqueos y acceso administrativo. Quedan fuera los pagos en línea, el inventario, la nómina, los recordatorios automáticos y las cuentas individuales de los barberos. Los horarios y las políticas que usa la demostración son supuestos académicos. Todavía falta acordarlos con un negocio real.

Diseño de la arquitectura

Separación de responsabilidades

El sistema sigue una arquitectura cliente-servidor. En el navegador, React muestra los formularios y la agenda. Cuando una persona reserva o modifica una cita, la solicitud llega a la API, que es la vía de comunicación con el servidor. Allí, Node.js y Express revisan las reglas y consultan MySQL. El navegador nunca accede por su cuenta a la base de datos.

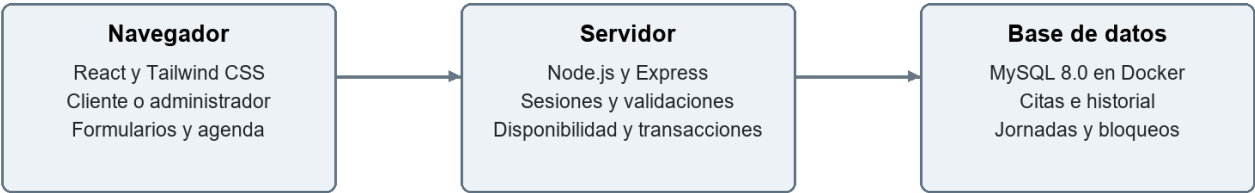


Figura 1. Comunicación entre la pantalla, el servidor y la base de datos. La interfaz intercambia datos por HTTP en formato JSON. El servidor usa consultas SQL con parámetros.

Componente	Responsabilidad	Elementos principales
Interfaz	Capturar datos y mostrar respuestas	Reserva, agenda, catálogos, jornadas, bloqueos y clientes
API	Autorizar y validar operaciones	Sesiones, rutas públicas y administrativas, reglas de disponibilidad
Persistencia	Guardar relaciones e historial	MySQL, claves foráneas, transacciones e índices

Organización de los módulos

El código del servidor está dividido según su tarea: revisar datos, gestionar el acceso, consultar la base o aplicar las reglas de la agenda. Cada pantalla administrativa también tiene su propio componente. Las pantallas comparten funciones para enviar solicitudes y mostrar mensajes o diálogos. Así, una corrección en esas funciones puede aprovecharse en varias pantallas.

Acceso y comunicación

Cualquier visitante puede consultar servicios, barberos y horarios libres, pero no los nombres ni los teléfonos de otros clientes. Para ver la agenda o hacer cambios se necesita una sesión vigente. Las contraseñas se procesan con `bcrypt` y una sal aleatoria, en lugar de guardarse como texto. La cookie de sesión usa `HttpOnly` y `SameSite Strict`. La referencia [3] explica la función `bcrypt` de Node.js.

Al ejecutar la versión local, la interfaz y la API usan la misma dirección. Durante el desarrollo, Vite envía las solicitudes de la API al servidor. Esto permite editar las pantallas mientras las reglas de las citas siguen a cargo del servidor.

Diseño de procesos y reglas

Un horario puede estar libre al consultarlo y ocuparse antes de confirmar. Por eso, el servidor lo revisa otra vez al guardar. Esa revisión y el registro de la cita forman parte de una transacción: si algo falla, la operación se revierte.

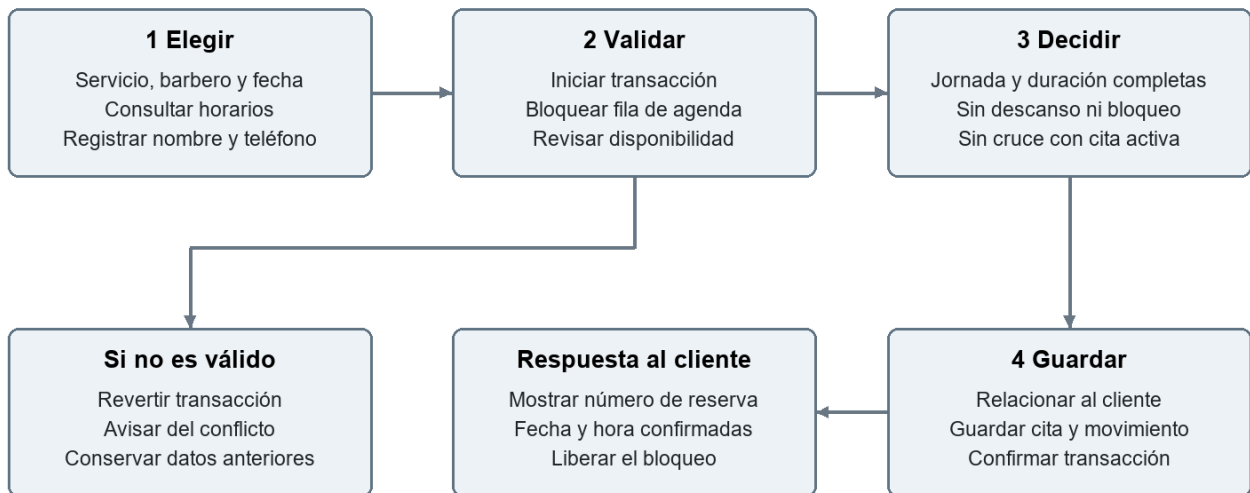


Figura 2. Pasos para reservar una cita. Si la validación falla, el proceso sigue la ruta inferior izquierda y devuelve un aviso al cliente.

Regla de conflicto

Dos citas se cruzan cuando $\text{inicio_nueva} < \text{fin_existente}$ y $\text{fin_nueva} > \text{inicio_existente}$. La regla se aplica al mismo barbero y a citas pendientes o confirmadas. En cambio, una cita que termina a las 10:40 permite que otra empiece a esa hora. Además, todo el servicio debe caber en la jornada, sin ocupar descansos ni periodos bloqueados.

Control de solicitudes simultáneas

Para evitar que dos solicitudes guarden el mismo horario, el servidor usa `SELECT FOR UPDATE` sobre una fila de control [2]. Mientras una operación revisa y guarda los datos, las demás esperan. Al continuar, vuelven a comprobar si el horario sigue libre. El bloqueo termina al confirmar o revertir la transacción. Este mecanismo ordena las escrituras, aunque también puede limitar la capacidad del sistema cuando aumente la demanda.

Cambios y estados

Al reprogramar, primero se revisa el nuevo horario. Si no es válido, la cita conserva sus datos. La cancelación libera el espacio y mantiene el historial. Una cita confirmada puede marcarse como completada después de terminar, o como ausencia después de su inicio. En la reserva pública, las citas quedan confirmadas desde el principio. El modelo admite el estado pendiente y sus cambios, pero el formulario público no permite crearlo. Este comportamiento ajusta el recorrido previsto en la primera entrega.

Diseño de la base de datos

Las tablas separan los catálogos de las citas y de los datos de acceso. Cada cita se vincula con un servicio y un barbero. Los registros nuevos también se vinculan con un cliente. Las claves foráneas mantienen estas relaciones. Cuando un servicio o barbero deja de estar disponible, se marca como inactivo para conservar los datos anteriores.

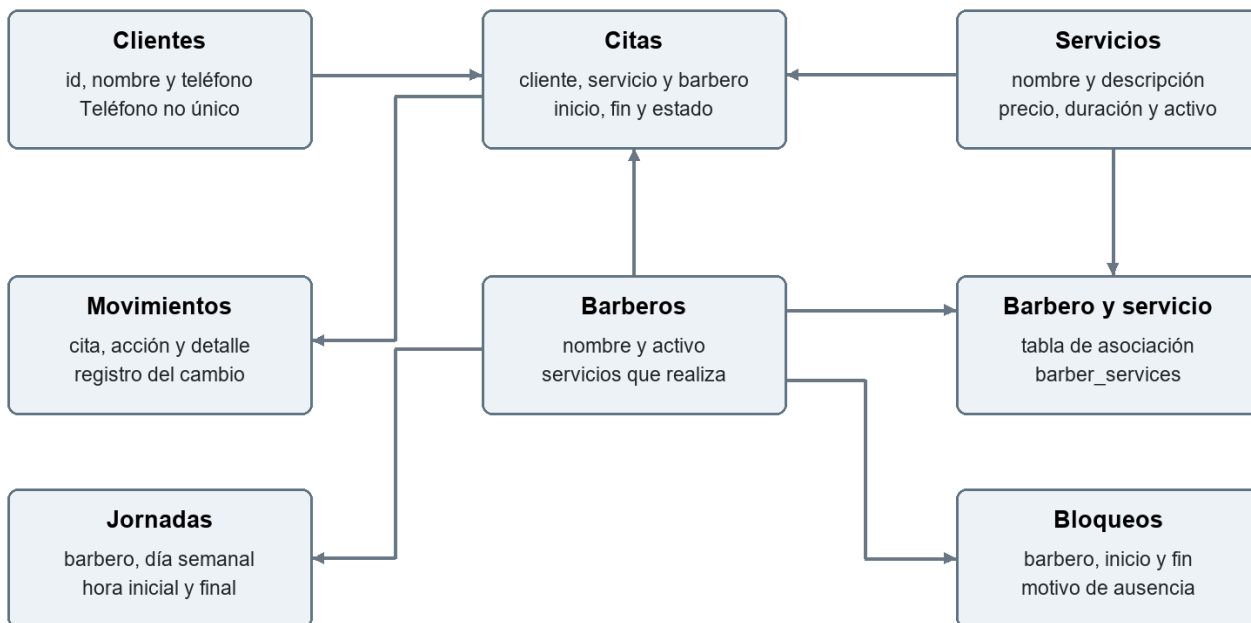


Figura 3. Relaciones de la base de datos. Las flechas indican qué registros dependen de otros. Por ejemplo, un cliente puede tener varias citas, y cada cita puede tener varios movimientos.

Decisiones de persistencia

Al reservar, el sistema guarda la hora de inicio y calcula la de fin. Si después cambia la duración del servicio, las citas anteriores conservan su horario. También guardan el nombre y el teléfono recibidos en la reserva. Un teléfono puede pertenecer a más de una persona. Si nombre y teléfono coinciden exactamente, se reutiliza el cliente registrado; cuando dos personas comparten ambos datos, el administrador debe revisar el caso.

Un barbero puede tener varios periodos de trabajo en el mismo día. El espacio entre ellos cuenta como descanso. Si un periodo termina justo cuando empieza otro, el sistema los trata como una jornada continua. Los bloqueos tienen inicio y fin, y pueden durar varios días. Cuando afectan una cita ya guardada, aparece un aviso para que el administrador la resuelva. La cita no se borra.

Otras tablas guardan administradores y sesiones (admins y admin_sessions), cambios de las citas (appointment_events), el bloqueo de agenda (schedule_mutex) y las actualizaciones de estructura (migrations). Los índices ayudan a consultar la agenda y buscar registros. Al iniciar, el sistema adapta la base anterior y vincula sus citas con clientes sin borrar la información existente.

Diseño de la interfaz

La pantalla conserva el fondo oscuro y los detalles dorados de Mala Vida Barbers. El cliente encuentra allí los servicios y elige barbero, fecha y hora. Luego escribe sus datos de contacto y confirma. Al terminar, recibe el número de reserva junto con el horario asignado.

MALA VIDA BARBERS

Tu estilo. Tu momento.

Elige tu servicio y encuentra un espacio con tu barbero. Nosotros nos encargamos del resto.

Corte Clásico \$250.00 40 min
Corte y acabado a tu estilo.

Corte y Barba \$350.00 60 min
Corte completo y arreglo de barba.

Diseño de Barba \$150.00 30 min
Perfilado y cuidado de barba.

Sin cuenta, sin complicaciones. Tu reserva se confirma al terminar.

AGENDA TU VISITA

Reserva tu cita

Servicio: Corte Clásico - 40 min - \$250.00 Barbero: QA Barbero

Fecha: 13/10/2026

Horarios disponibles

10:00	10:10	10:20	10:30	10:40	10:50	11:00
11:10	11:20	11:30	11:40	11:50	12:00	12:10
12:20	12:30	12:40	12:50	13:00	13:10	13:20
15:00	15:10	15:20	15:30	15:40	15:50	16:00
16:10	16:20	16:30	16:40	16:50	17:00	17:10

Hora de Ciudad de México. Reservas con hasta 90 días de anticipación.

Nombre completo: Cliente Navegador Teléfono: 5598765432

Tus datos se usan para gestionar tu cita y solo son visibles para el administrador.

10:00 - Corte Clásico \$250.00

Confirmar reserva

MALA VIDA BARBERS Agenda con confianza. Llegá con estilo.

Figura 4. Pantalla de reserva durante una prueba automática. El nombre y el teléfono que aparecen son datos de prueba.

Administración y prevención de errores

El panel tiene apartados para agenda, clientes, servicios, barberos, jornadas, bloqueos y cuenta. Para encontrar una cita se puede filtrar por fecha, barbero, estado, nombre o teléfono. Antes de cancelar o cambiar el estado, el sistema pide confirmar la acción. Después informa si pudo guardarla o si hubo un conflicto. También avisa cuando un bloqueo o un cambio de jornada afecta citas existentes.

En el teléfono, los formularios y las tarjetas aparecen en una sola columna. La revisión a 390 por 844 píxeles no mostró desplazamiento horizontal. Los campos tienen etiquetas, el foco del teclado es visible y los estados se leen en texto, además de distinguirse por color. Aún falta evaluar la accesibilidad con más detalle y observar cómo usan el sistema personas reales.

Desarrollo e implementación

Tecnologías y construcción

La interfaz usa React, Vite y Tailwind CSS. El servidor está escrito en JavaScript con Node.js y Express, y se conecta a MySQL mediante mysql2. Docker guarda la base en un volumen para conservarla entre reinicios. Las funciones del proyecto se agrupan en los apartados que muestra la siguiente tabla.

Incremento funcional	Resultado implementado
Reservas y disponibilidad	Calcula horarios válidos y rechaza cruces, incluso ante solicitudes simultáneas.
Administración	Permite iniciar sesión, filtrar citas, cambiar su horario, cancelarlas y actualizar su estado.
Configuración	Permite asignar servicios a los barberos y definir sus jornadas y bloqueos.
Seguimiento y verificación	Conserva clientes e historial e incluye pruebas de la API y del navegador.

El plan de la primera entrega incluía cuatro iteraciones de dos semanas y proponía usar Trello, Figma y Visual Paradigm. Para esta entrega se cuenta con el código, los diagramas del informe y los registros de pruebas. Esos materiales permiten revisar el trabajo realizado, aunque no acreditan fechas de cada iteración, entrevistas ni el uso de todas las herramientas propuestas.

Integración y conservación de datos

Al iniciar, el sistema crea las tablas que faltan y actualiza la estructura anterior. Deja un registro de esa actualización para no repetir la carga. Antes de guardar, revisa identificadores, fechas, teléfono y duración, y usa parámetros en las consultas. Los cambios de agenda comparten el mismo control de transacciones. Una prueba comprobó que iniciar de nuevo no duplicara servicios ni jornadas.

Configuración de demostración

La demostración usa la hora de Ciudad de México. El horario inicial va de lunes a sábado, de 10:00 a 14:00 y de 15:00 a 20:00, aunque puede cambiarse para cada barbero. El cliente puede reservar hasta con 90 días de anticipación y ve opciones cada 10 minutos. Para dar de alta un barbero hay que asignarle una jornada. También se debe indicar qué barberos realizan cada servicio nuevo.

El sistema se abre localmente en <http://127.0.0.1:3001>. El archivo README del proyecto explica cómo instalarlo y ponerlo en marcha. Falta publicar el repositorio remoto y añadir su enlace a la entrega. La dirección local solo sirve en el equipo donde se ejecuta la aplicación.

Pruebas y resultados

Método de verificación

Las pruebas que respaldan este informe se ejecutaron el 23 de septiembre de 2026 en Windows, con Node.js 24.19.0, MySQL 8.0.45 y Microsoft Edge 153.0.4234.48. Usaron datos ficticios en una base temporal y enviaron solicitudes HTTP al servidor. Playwright 1.58.2 automatizó las acciones en el navegador. Al terminar, se borró únicamente la base creada para esas pruebas.

Aprobaron cuatro casos de reglas básicas y 22 de integración: 26 casos en total. El registro cuenta además la aprobación del grupo de integración, por lo que muestra 27 resultados. La tabla resume las comprobaciones principales. Los casos y sus resultados pueden consultarse en los archivos de pruebas del proyecto.

Comprobación	Resultado observado
Solicitudes simultáneas	20 intentos sobre un mismo intervalo: 1 reserva creada y 19 rechazos por conflicto
Reprogramación inválida	El cambio se rechazó y la cita conservó su identificador y su hora de inicio.
Cancelación	La cita siguió en el historial y el horario volvió a estar libre.
Permisos	El servidor rechazó las consultas sin sesión con la respuesta HTTP 401.
Bloqueos y jornadas	El sistema excluyó los periodos ocupados y avisó de las citas afectadas.
Recorrido en navegador	Reserva, acceso, reprogramación, cancelación, historial y formularios administrativos completados
Resultado global	27 resultados aprobados, ninguno fallido ni omitido. Duración aproximada: 14.32 segundos.

Rendimiento observado

Se midieron 20 consultas de disponibilidad, una después de otra, con conexión local y una base pequeña. La mediana fue de 16.5 ms y el valor más alto, de 25.9 ms. Ambos quedaron por debajo de los 2 segundos establecidos en RNF-04. Estos tiempos corresponden a la respuesta de la API. Falta medir la carga completa de la página y el comportamiento con más usuarios.

Correcciones verificadas

Durante las pruebas se corrigieron las etiquetas que permiten identificar los selectores y el manejo de periodos de trabajo consecutivos. También terminaron sin errores ni advertencias la compilación y el análisis estático de la interfaz. Los casos revisados funcionaron como se esperaba, aunque pueden existir fallos en situaciones que todavía no se han probado.

Evaluación de calidad

La revisión de calidad usa ISO/IEC 25010:2023 como referencia general [4]. La tabla reúne los aspectos que se pudieron revisar en este prototipo y señala lo que falta comprobar. Esta evaluación local no equivale a una certificación ni a una revisión completa de la norma.

Aspecto	Criterio y evidencia	Valoración
Adecuación funcional	Las pruebas revisan las reservas y los cambios de las citas.	Las funciones principales están listas. La reserva pública confirma directamente.
Fiabilidad e integridad	Se probaron cruces, cambios rechazados, solicitudes simultáneas e historial.	Los casos aprobaron. Falta probar fallos prolongados.
Eficiencia	En 20 consultas locales: mediana de 16.5 ms y máximo de 25.9 ms.	Se cumplió la meta local de 2 s. Falta probar con mayor carga.
Seguridad	Exige sesión y usa script, opciones de protección de cookies y consultas con parámetros.	Se revisaron controles básicos. Falta una prueba de penetración.
Interacción y adaptación	Incluye etiquetas, avisos y confirmaciones. La vista móvil cabe en la pantalla.	La revisión técnica fue favorable. Falta probar con usuarios.
Mantenibilidad	El código separa módulos, comparte componentes y permite repetir pruebas.	La estructura facilita revisar el código. No se midió su cobertura.
Compatibilidad del entorno	Se usó Edge en la prueba final y Chrome durante el desarrollo.	Falta comprobar Firefox y Safari.

Métricas y alcance de los resultados

Los 26 casos ejecutados aprobaron, así que la tasa de aprobación fue del 100 %. Ese dato describe esta ejecución y no indica qué porcentaje del código o de los requisitos quedó cubierto. En el intento simultáneo de 20 reservas para el mismo horario, solo una se guardó. Las pantallas revisadas también se mantuvieron dentro del ancho disponible.

Límites y siguientes validaciones

Antes de usar el sistema en una barbería, hace falta acordar con el encargado las reglas de anticipación, cancelación y retrasos. Después conviene probarlo con clientes, más registros y otros navegadores, además de revisar su accesibilidad. Publicarlo requiere HTTPS, credenciales de base de datos adecuadas y un procedimiento de respaldo y recuperación. Estos pasos corresponden a la continuación del proyecto y no modifican la primera entrega.

Conclusiones y referencias

Conclusiones

La segunda etapa deja un prototipo con el que se pueden reservar y administrar citas. La disponibilidad considera la duración del servicio y los horarios del barbero, incluidos sus descansos y bloqueos. Al guardar, el servidor vuelve a revisar esas condiciones para evitar aceptar un espacio que ya se ocupó.

Los cambios de horario y las cancelaciones quedan en el historial. Esto permite consultar qué pasó con una cita, incluso si un intento de cambio fue rechazado. Además, la configuración de cada barbero determina qué servicios puede atender y en qué horarios aparecen disponibles.

Las pruebas realizadas respaldan el uso del prototipo en una demostración académica. Para conocer su utilidad en la operación diaria, todavía hay que probarlo con un encargado y con clientes. Por ahora no se ha medido una reducción de ausencias, la satisfacción de quienes lo usan ni su funcionamiento continuo en un entorno público.

Referencias

[1] INGE00031. (s. f.). Producto Transversal. Ingeniería de Software. Documento de la actividad, páginas 4 a 6.

[2] Oracle. (s. f.). MySQL 8.0 Reference Manual. Locking Reads. <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html>

[3] Node.js. (s. f.). Crypto. crypto.scrypt. <https://nodejs.org/api/crypto.html#cryptoscriptpassword-salt-keylen-options-callback>

[4] ISO e IEC. (2023). ISO/IEC 25010:2023. Systems and software engineering. Systems and software Quality Requirements and Evaluation. Product quality model. Ficha pública del estándar. <https://committee.iso.org/standard/78176.html>

Fuentes web consultadas el 20 de septiembre de 2026.

Uso de herramientas de apoyo

Se usó inteligencia artificial como apoyo para implementar y revisar el código, y para preparar y revisar la redacción del informe.

Los resultados descritos se basan en el código y en las pruebas del proyecto. No se realizaron entrevistas ni pruebas con clientes reales.